POLITECNICO DI TORINO

Master degree in Cybersecurity

Master Degree Thesis

An incremental approach to automatic firewall configuration

Supervisor
prof. Fulvio Valenza
prof. Daniele Bringhenti
prof. Francesco Pizzato

Candidate
Filippo Del Ministro

Academic Year 2025-2026

Nowadays, computer networks are complex systems. Modern networks are in fact often distributed systems in which security policy enforcement is a critical and continuous operational challenge. As topologies grow in scale and diversity, the manual configuration of packet-filtering firewalls becomes increasingly prone to errors. Misconfigurations can expose sensitive resources or degrade service availability. In this context, the shift toward microsegmentation, distributing enforcement across multiple firewall instances throughout the network, improves security granularity but amplifies the management effort. Automated tools that generate formally correct and optimal configurations are therefore essential.

VEREFOO (VErification REasoning on Firewall Optimization) addresses this need by automatically computing firewall allocation and configuration from a network topology and a set of Network Security Requirements (NSRs). It encodes the problem as a formal optimisation problem — specifically a Maximum Satisfiability Modulo Theories (MaxSMT) instance. This approach provides two key guarantees: formal correctness, meaning that the generated configuration provably satisfies every specified requirement, and minimality, meaning that the number of allocated firewall instances and configured rules is kept as low as possible. However, VEREFOO follows a monolithic approach: every time the NSR set changes, the entire problem is re-encoded and solved from scratch, scaling poorly as the network or requirement set grows.

React-VEREFOO addresses this by taking an incremental approach. It identifies which requirements are new, removed, or unchanged, and restricts the solver to only the affected parts of the network. This reduces reconfiguration cost when few requirements change, but the benefit decreases when a large fraction of the NSR set is modified simultaneously.

This thesis pushes the incremental principle to its extreme. The proposed Iterative React-VEREFOO introduces new NSRs one at a time: at each step, the current configuration is used as the starting state and exactly one requirement is added. React-VEREFOO computes the minimal update, and the result becomes the input to the next iteration. By construction, the reconsidered portion of the network is always minimal, keeping each MaxSMT subproblem small regardless of the total number of requirements.

The approach is evaluated on a set of benchmark topologies covering both real-world and synthetic networks of varying scale, with the aim of quantifying whether the consistent reduction in per-step problem size translates into a measurable improvement in total execution time compared to the monolithic VEREFOO baseline.

Index

Chapter 1

Introduction

Modern enterprise networks are large, dynamic infrastructures that change continuously. New services are deployed, nodes are added or removed, and connectivity requirements evolve at a pace that security policy management must match. At the same time, these networks reach hundreds of heterogeneous nodes distributed across sites and organisational boundaries, making security enforcement a complex and ongoing operational challenge [1]. Packet-filtering firewalls are the primary mechanism for enforcing security policies, and their correct configuration is essential to prevent unauthorized access, data breaches, and service disruptions.

The traditional approach to firewall configuration relies on manual translation of high-level security requirements into device-level rules, and this approach does not scale to modern network environments. As networks grow in size and change more frequently, the volume and complexity of rules to be managed quickly exceeds human capacity. More critically, manual processes offer no formal guarantee of correctness: a single misconfiguration may silently violate security requirements or block legitimate traffic, with consequences that can remain undetected for extended periods.

VEREFOO was developed to address this problem through full automation, ensuring formal correctness and optimality. Given a network topology and a set of Network Security Requirements (NSRs), it automatically determines where firewalls should be placed and what filtering rules each should enforce [2]. It achieves this by encoding the problem as a formal optimisation problem, namely a Maximum Satisfiability Modulo Theories (MaxSMT) instance, solved by the Z3 solver. However, VEREFOO follows a monolithic strategy: any change to the NSR set triggers a complete recomputation from scratch, making it poorly suited to dynamic environments where requirements change frequently such as modern enterprise networks.

React-VEREFOO addressed this limitation by adopting an incremental approach [3]. Rather than discarding the existing configuration, it identifies which NSRs are new, which have been removed, and which are unchanged, and restricts the solver to only the affected parts of the network. This yields significant speedups when the number of modified requirements is small. However, when a large fraction of the NSR set changes simultaneously, the advantage in terms of execution time is reduced and performance results similar to the monolithic baseline.

This thesis proposes a strategy that preserves the benefits of the incremental approach regardless of how many novel requirements are introduced. The key idea is to push the incremental principle to its extreme: the iterative approach proposes to perform a full configuration from scratch by introducing new NSRs one at a time, invoking React-VEREFOO internally at each step to compute the minimal update on the current configuration. By construction, the modified fraction at each step is always minimal — one single requirement — so the subproblem remains small and the performance advantage of the incremental approach is consistently in effect.

The central research question this thesis aims to answer is whether invoking React-VEREFOO multiple times during the iterative approach yields a lower total execution time than vanilla VEREFOO, which solves the same problem in a single monolithic call. The intuition is that many cheap solver calls may collectively outperform a single expensive one; whether this holds in practice, and under what conditions, is the empirical question addressed in Chapter 5. The full formulation of the approach and the thesis objectives are presented in Chapters 3 and 4.

The remainder of the thesis is organised as follows. Chapter 2 provides the necessary background on network security automation, VEREFOO, and React-VEREFOO. Chapter 3 states the thesis objectives and introduces the iterative approach at a high level. Chapter 4 develops the approach from a theoretical perspective. Chapter 5 describes the implementation and reports the experimental evaluation on three benchmark topologies: the Stanford university network, the CESNET academic network, and the Texas 2000 synthetic topology. Chapter 6 draws conclusions and outlines directions for future work.

Chapter 2

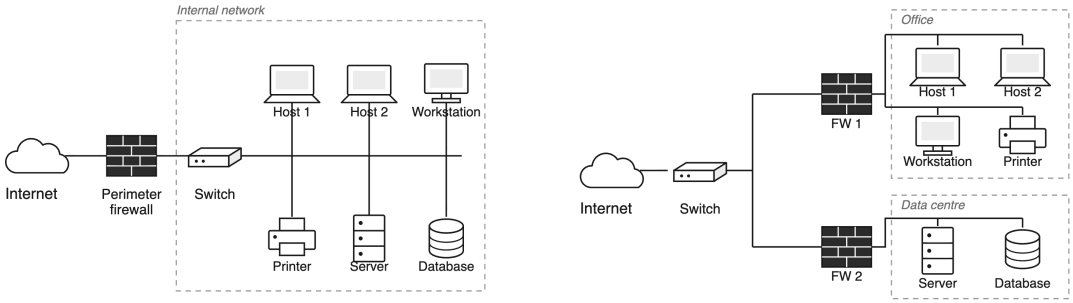
Background

2.1 Context

Network security is a critical concern for mostly every organization, from small businesses to large enterprises and public institutions. All of them rely on computer networks to support their operations, and all of them are exposed to the risk of unauthorized access, data breaches, and service disruptions. As a result, enforcing a well-defined and correct security policy is not optional, but instead is a fundamental operational requirement.

Modern enterprise networks are characterized by growing complexity: hundreds of interconnected nodes, heterogeneous services, and security policies that must be consistently enforced across distributed infrastructure. Moreover, network topologies are not static: new services are deployed, nodes are added or removed, and connectivity requirements evolve continuously, making security management an ongoing and demanding task. The cost of managing network security in this context is significant, as it requires specialized expertise, advanced planning, and constant monitoring.

A central challenge in this landscape stems from the evolution of network security architectures. Traditionally, a single perimeter firewall was sufficient to protect a network by controlling traffic at its boundary. However, as networks grew in scale and complexity, this model was proved as inadequate: a single point of control cannot enforce fine-grained policies across different internal topology. Modern networks have therefore evolved toward microsegmentation, deploying multiple packet-filtering firewall instances strategically distributed across the topology to enforce security policies at a much more granular level , as illustrated in Figure 2.1. This distributed approach offers clear advantages such as enabling finer-grained traffic control, reduces single points of failure, and allows security policies to be enforced closer to the traffic sources. On the other hand, this approach also introduces considerable management complexity. Each firewall instance must be individually configured with a set of filtering rules, and those rules must collectively enforce the intended security policy across the entire network, without conflicts or gaps.



(a) Traditional perimeter firewall architecture. A single point of control at the network boundary; internal traffic is unfiltered.

(b) Distributed firewall architecture (microsegmentation). A filtering instance guards each segment, so internal traffic is filtered as well.

Figure 2.1: Evolution of network security architectures: from a single perimeter firewall to distributed packet-filtering instances deployed across the topology.

In this context, the configuration of firewalls becomes a critical task. On one hand, overly permissive rules may allow unauthorized traffic to reach sensitive resources, exposing the network to attacks. On the other hand, overly restrictive rules may block legitimate traffic, degrading service availability and causing operational disruptions. Both failure modes carry significant consequences, and yet both are commonly caused by manual configuration process which is inherently prone to errors especially for large-scale networks [4].

The gap between the complexity of modern networks and the limitations of manual security management calls for automated approaches that can generate correct and optimal firewall configurations without human intervention. It is in this context that VEREFOO was developed: a tool for the automated, formally correct allocation and configuration of distributed firewalls, designed to bridge exactly this gap.

2.2 VEREFOO

2.2.1 Overview

VEREFOO (VErification REasoning on Firewall Optimization) is a tool designed to automate the allocation and configuration of packet-filtering firewalls in a network, eliminating entirely the need for manual intervention [2]. As exemplified in Figure 2.2, given a network topology and a set of Network Security Requirements (NSRs) — high-level specifications describing which traffic flows must be allowed or denied between network endpoints [5] — VEREFOO automatically determines where firewalls should be placed within the topology and generates the corresponding filtering rules for each allocated instance. The resulting configuration is expressed in a vendor-independent format, which can then be translated into concrete rules for specific implementations such as iptables, BPF, or OVS.

The tool provides three formal guarantees. *Automation* ensures that no human intervention is required at any stage of the process. *Formal correctness* guarantees that the generated configuration satisfies all specified NSRs by construction, eliminating the risk of undetected misconfigurations. *Optimality* ensures that both the number of allocated firewall instances and the number of generated rules are minimised, reducing resource consumption and improving performance. These properties are achieved by encoding the firewall allocation and configuration problem as a partial weighted Maximum Satisfiability Modulo Theories (MaxSMT) instance, which is then solved using the Z3 SMT solver. The hard constraints of the problem encode the correctness requirements, while the soft constraints encode the optimality objectives; the solver finds a solution that satisfies all hard constraints and maximises the sum of satisfied soft constraint weights.

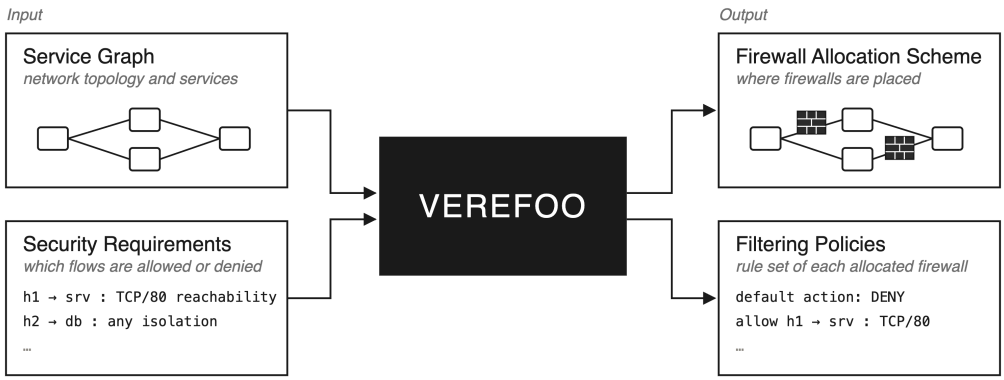


Figure 2.2: High-level view of VEREFOO: given the Service Graph and the Network Security Requirements, the tool produces the firewall allocation scheme and the filtering policy for each allocated instance guaranteeing formal correctness and optimality.

Before the MaxSMT problem is formulated, VEREFOO must compute the set of traffic flows to be modelled. Two algorithms are available for this step. The *Maximal Flows* algorithm (MF) or *Atomic Predicates* algorithm (AP). The MF algorithm groups packets into maximal flows, which are flows that are not subflows of any other flow; in other words, multiple packets with identical traversal behaviour are represented by a single entity, reducing the number of constraints in the problem [2]. The Atomic Predicates instead, goes further by decomposes every traffic predicate into a set of atomic predicates that are minimal, unique, and mutually disjoint, then expresses each flow in terms of these atomic predicates. This additional decomposition reduces the number of distinct cases the solver must consider and has been shown to provide better performance for automatic (re)configuration problems [3].

2.2.2 Network Security Requirements

Network Security Requirements (NSRs) are the high-level specification of the security policy that a network must enforce. Rather than describing firewall rules directly, NSRs express what the administrator intends: which traffic flows between pairs of endpoints must be permitted and which must be denied. This level of abstraction decouples the intent of the security policy from its low-level implementation, allowing VEREFOO to take full responsibility for translating requirements into concrete filtering rules.

Each NSR refers to a specific traffic flow, identified by five attributes: source address, destination address, transport protocol, source port, and destination port. The administrator specifies, for each such flow, whether it must be reachable (i.e., allowed end-to-end through the network) or isolated (i.e., blocked). VEREFOO supports four different policy modes that determine how the requirements are interpreted: *whitelisting*, where all traffic is blocked by default and only explicitly allowed flows are permitted; *blacklisting*, where all traffic is allowed by default and only explicitly listed flows are denied; and two mixed modes (*rule-oriented specific* and *security-oriented specific*), which accept both reachability and isolation requirements without a fixed default behavior, differing in how unspecified flows are handled [2].

NSRs are provided to VEREFOO as part of its input, alongside the network topology. Internally, they are represented and stored as a structured set of logical constraints, expressed in a vendor-independent intermediate language. A higher-level specification language is also supported, from which the corresponding medium-level NSRs are automatically derived through a translation step. VEREFOO processes this set together with the Allocation Graph derived from the topology, and encodes each NSR as a hard constraint in the MaxSMT problem: the solver is required to find a firewall allocation and rule set that satisfies all of them without exception. If no valid solution exists — for example because user-defined constraints on the Allocation Graph have excluded all possible firewall positions — VEREFOO produces a non-enforceability report rather than a configuration.

Table 2.1 shows three representative NSRs illustrating how requirements of different types are expressed.

Table 2.1: Representative NSR examples.

Type	**Src**	**Dst**	**Protocol**	**Port**	**Description**
Reachability	40.40.41.1	130.10.0.1	any	any	trusted client may reach server
Isolation	40.40.42.1	130.10.0.1	any	any	untrusted client is blocked
Reachability	40.40.41.1	130.10.0.1	TCP	80	allow TCP traffic on port 80

2.2.3 Service Graph

A Service Graph (SG) is the logical representation of a virtual network: a directed graph in which nodes represent Network Functions (NFs) such as web servers, load balancers, caches, or client subnetworks while edges represent the connectivity between them. The SG captures all possible end-to-end traffic paths through the network at an abstract level, without exposing low-level forwarding infrastructure such as switches and routers. It is defined taking into consideration solely the services that the network must provide, independently of any security consideration [2].

The SG serves as the structural input to VEREFOO: it defines the space of possible firewall positions and the traffic paths that any allocated firewall must be able to intercept. To make this explicit, VEREFOO automatically derives an internal representation called the *Allocation Graph* (AG) from the SG. For each link between any pair of nodes in the SG, the AG introduces a placeholder element called an *Allocation Place* (AP) which represents a candidate position where a firewall instance may be allocated. Those Allocation Places are then used by VEREFOO during the optimal placement of firewalls. Optionally, Verefoo allow to manually constrain this process by forcing a firewall into a specific AP or excluding certain APs from consideration, for example to model pre-existing hardware firewalls that cannot be moved. Figure 2.3 illustrates how a simple Service Graph is transformed into its corresponding Allocation Graph.

VEREFOO uses the Allocation Graph (AG) jointly with the Network Security Requirements (NSRs) to formulate the MaxSMT problem. Each AP corresponds to boolean decision variable that the solver uses to determines the optimal firewall placement and what filtering rules each firewall should enforce, such that all NSRs are satisfied and the total number of allocated firewalls and rules is minimised.

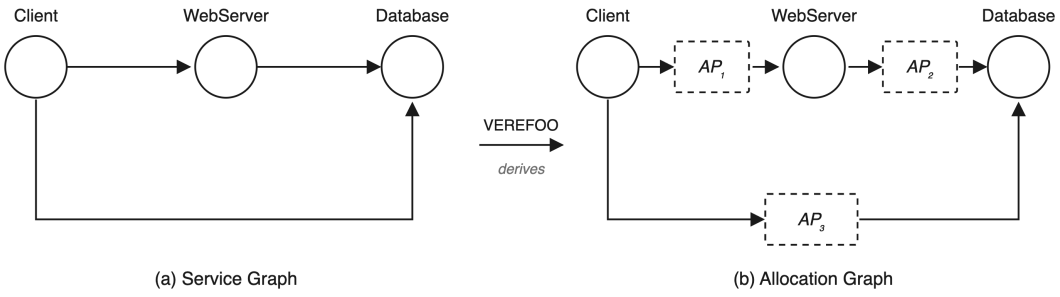


Figure 2.3: From Service Graph (left) to Allocation Graph (right). VEREFOO automatically derives the Allocation Graph by inserting an Allocation Place (shown as dashed boxes) on each link, representing a candidate position where a firewall instance may be allocated by the solver.

2.2.4 VEREFOO Output

When the MaxSMT problem is solved successfully, VEREFOO produces two complementary outputs [2]. The first is the *firewall allocation scheme*: an updated version of the Allocation Graph in which each selected Allocation Place hosts an active firewall instance, with the minimum number of firewalls necessary to enforce all NSRs. The second is the *Filtering Policy* (FP) for each allocated firewall: a complete, auto-generated rule set that defines how the firewall must handle every traffic flow passing through it. Each FP is expressed in a vendor-independent abstract language, making it independent of any specific firewall implementation. It consists of a default action (either drop-all in whitelisting mode, or allow-all in blacklisting mode) and a minimal set of explicit rules covering the traffic flows specified by the NSRs. Both the default action and the explicit rules are guaranteed to be anomaly-free, with no conflicts or redundancies among them.

Together, these two outputs constitute the full security configuration of the network at the logical level of the Service Graph, independent of the physical infrastructure. A subsequent translation step converts the abstract filtering policies into concrete rules for specific implementations such as iptables, BPF, or OVS [2]. If instead no solution exists, for example because user-defined constraints on the AG have excluded all viable firewall positions, VEREFOO returns a non-enforceability report in place of the configuration.

2.2.5 Verefoo validation

VEREFOO has been validated on both synthetic and real-world network topologies, including well-known benchmarks such as the Stanford and CESNET networks, demonstrating reliability and scalability across a range of network sizes and requirement sets [2].

Despite these strengths, VEREFOO's monolithic approach has a fundamental scalability limitation: the entire set of NSRs and the full network topology are encoded into a single MaxSMT problem. As the number of requirements or the size of the network grows, the complexity of the problem increases significantly, leading to prohibitive computation times. This limitation motivates the development of more efficient alternatives, starting with React-VEREFOO.

Further details on the experimental validation of VEREFOO and its comparison with the iterative approach proposed in this thesis can be found in Chapter 5.

2.3 React-VEREFOO

In operational environments, network security policies are not static: NSRs evolve over time as new services are deployed, topology changes occur, or threat models are updated. A fundamental limitation of the monolithic approach used by VEREFOO is that any change to the NSR set triggers a full recomputation of the firewall configuration from scratch. React-VEREFOO addresses this by offering a more efficient approach in the dynamic context described above, avoiding the need to

discard a valid existing configuration when only a small portion of the requirements has changed.

React-VEREFOO takes two inputs: the existing firewall configuration (allocation scheme and filtering rules) and a pair of NSR sets — the *initial set* R_i, describing the requirements already enforced by the current configuration, and the *target set* R_t, describing the requirements that must be enforced after reconfiguration. The approach proceeds in three steps.

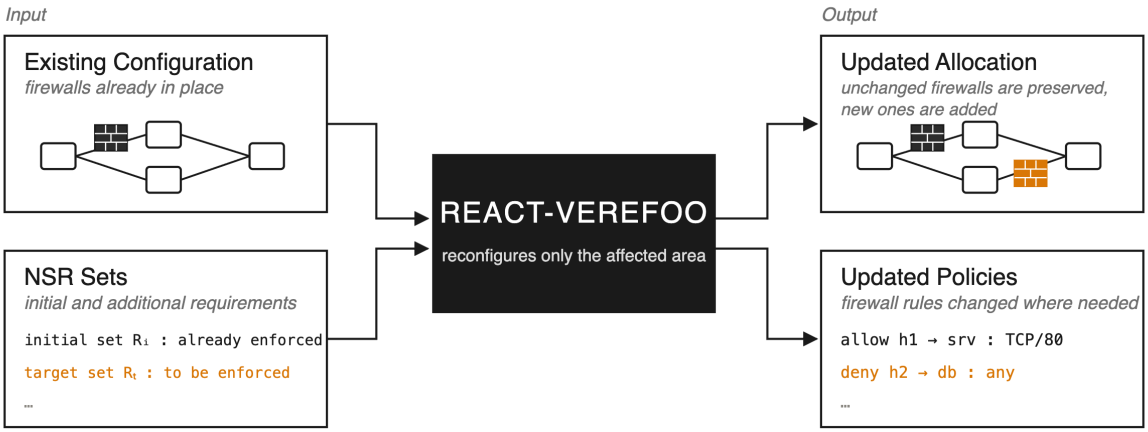


Figure 2.4: High-level view of React-VEREFOO: given the existing configuration, the initial and the updated NSR sets, it classifies requirements into added, kept, and deleted groups, identifies the minimal network area affected by the added requirements, and solves a reduced MaxSMT problem restricted to that area, leaving the rest of the configuration untouched.

Step 1 — NSR classification. React-VEREFOO partitions every NSR into one of three groups:

- $R_d = \{r \in R_i \mid r \notin R_t\}$: *deleted* requirements, previously enforced but no longer needed. They are simply excluded from the new problem formulation.
- $R_a = \{r \in R_t \mid r \notin R_i\}$: *added* requirements, new NSRs that must be enforced but are not yet satisfied. These are the sole driver of reconfiguration.
- $R_k = \{r \in R_t \mid r \in R_i\}$: *kept* requirements, already correctly enforced and still valid. The nodes satisfying them are left unchanged.

Step 2 — Identification of the network area to reconfigure. For each added requirement in R_a, React-VEREFOO analyses the Allocation Graph to determine which nodes are in conflict with the new requirement and therefore need to be reconsidered. The procedure differs depending on whether the added requirement is an isolation or a reachability constraint [3].

For an added *isolation* requirement (traffic that must be blocked), the algorithm examines all atomic flows associated with that requirement. If a flow is already blocked by an allocated firewall, no action is needed for that flow. If instead no node along the path currently blocks the traffic, all nodes on that path are flagged as eligible for reconfiguration, since one of them must be configured to enforce the isolation.

For an added *reachability* requirement (traffic that must be allowed), the algorithm checks whether at least one atomic flow reaches the destination without being blocked. If such a flow already exists, the requirement is already satisfied and no reconfiguration is needed. Otherwise, the nodes that are currently blocking the traffic are flagged as eligible for reconfiguration.

In both cases, the algorithm selects all nodes that *could potentially* be used to fulfill the new requirement, since the optimal choice among them is left to the solver. Nodes not flagged by any added requirement retain their current configuration and are excluded from the decision variables of the subsequent MaxSMT problem.

An example of this behaviour for both cases is shown in Figure 2.5. The topologies in this example are purely illustrative and should not be considered realistic network scenarios; they are presented solely to clarify what happens internally when a new isolation or reachability requirement is introduced.

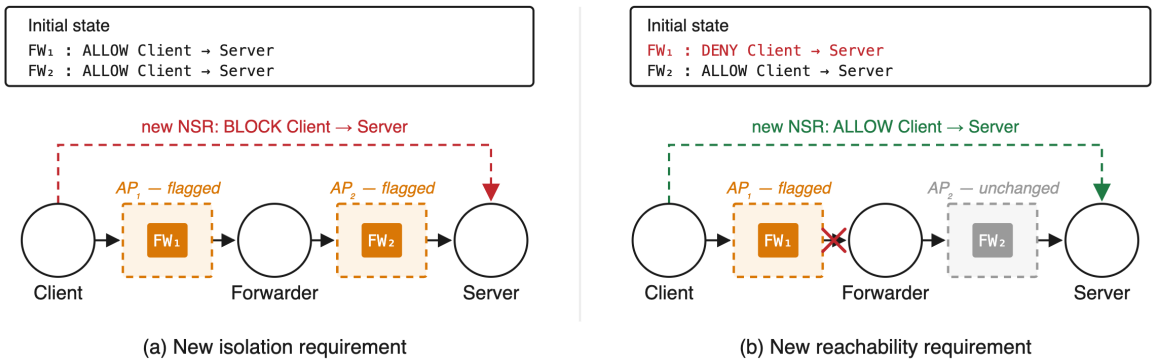


Figure 2.5: React-VEREFOO Step 2: example of identification of the network area to reconfigure, depending on the type of added NSR. **(a) New isolation requirement:** both FW_1 and FW_2 are initially configured to allow Client→Server traffic. Since no node currently blocks the flow, all nodes on the path are flagged as candidates for reconfiguration, and one of them must be configured to enforce the new requirement. **(b) New reachability requirement:** FW_1 is initially configured to deny Client→Server traffic, while FW_2 allows it. Since FW_1 is the node actively blocking the flow, only it is flagged for reconfiguration while FW_2 is left unchanged.

Step 3 — MaxSMT problem formulation and solving. Once the reconfigurable area has been identified, React-VEREFOO formulates a MaxSMT problem. This problem models traffic flows for all NSRs in the target set, ensuring that the final configuration is correct with respect to all requirements. However, only the

flagged nodes appear as free decision variables; the configuration of all other nodes is treated as fixed. This restriction dramatically reduces the size of the search space compared to a full recomputation.

The soft constraints are also adjusted to bias the solver toward reusing the existing configuration: allocating an already-deployed firewall is preferred over placing a new one, and retaining an existing rule is preferred over generating a new one. This minimises the number of changes applied to the network, reducing the time needed to deploy the new configuration.

The result is an updated allocation scheme and a revised set of filtering policies that satisfy all requirements in the target set, with formal correctness guaranteed by construction. As with VEREFOO, the output may not be globally optimal, since only a subset of the network is reconsidered, but it is optimal within the identified reconfiguration area.

Experimental results [3] confirm that React-VEREFOO is significantly more efficient than vanilla VEREFOO when the proportion of modified NSRs is small, as the reduced problem size leads to much shorter solving times. However, when a large fraction of the requirements changes, the benefit is reduced and performance converges toward the monolithic baseline. This observation motivates the approach investigated in this thesis: by pushing the incremental logic to its extreme and introducing only one new NSR per iteration. The proportion of modified requirements is always minimised, potentially achieving consistent speedups regardless of the total number of requirements.

Chapter 3

Thesis Objectives

3.1 Idea of the Solution

To understand the motivation of this thesis, it is necessary to examine how React-VEREFOO determines what to reconfigure. Given an initial set of NSRs R_i already enforced in the network and a target set R_t that must be enforced after reconfiguration, React-VEREFOO classifies every requirement into one of three groups [3]:

- $R_d = \{r \in R_i \mid r \notin R_t\}$: *deleted* requirements, present in the initial configuration but no longer needed;
- $R_a = \{r \in R_t \mid r \notin R_i\}$: *added* requirements, new requirements that must be enforced but are not yet satisfied;
- $R_k = \{r \in R_t \mid r \in R_i\}$: *kept* requirements, already enforced and still valid in the target configuration.

Only the added requirements R_a drive the reconfiguration: React-VEREFOO identifies which nodes in the Allocation Graph are in conflict with each added NSR and restricts the solver to those areas, while leaving the rest of the configuration untouched. The final MaxSMT problem models all traffic flows for the target set $R_t = R_a \cup R_k$, but the decision variables are limited to the nodes identified as needing reconfiguration, making the problem significantly smaller than a full recomputation.

The key performance parameter is the proportion of kept requirements, which can be defined as:

$$\text{PercReqKept} = \frac{R_k}{R_k + R_a + R_d}$$

The higher this ratio, the smaller the added and deleted sets, and consequently the narrower the network area that must be reconsidered. Experimental results in [3] confirm that the approach yields its greatest speedups at high values of PercReqKept, and that performance converges toward the vanilla baseline as PercReqKept decreases. This is consistent with real-world operational patterns: studies of large-scale infrastructure [3] report that the vast majority of configuration updates affect

only a small fraction of the total rule set, making high PercReqKept the typical case in practice.

The key idea of this thesis is to push this observation to its logical extreme. Instead of processing all new NSRs in a single reconfiguration step, the proposed approach introduces them **one at a time**. At each step, only one new requirement is presented to React-VEREFOO as added, while everything enforced so far falls into the kept set. This means the reconfiguration area identified at each step is always the smallest it could possibly be — a single requirement drives each solver call, regardless of how many requirements must ultimately be incorporated. The proportion of kept requirements is therefore always at its theoretical maximum for the given state of the network, and grows larger with every iteration as the kept set expands.

Concretely, the process unfolds as a chain of reconfiguration steps. The network starts from an existing valid configuration. One new requirement is then selected and processed by the iterative system, which internally invokes React-VEREFOO to compute the minimal update to the current configuration. The result, the new allocation scheme and updated filtering rules become the starting point for the next step. Another requirement is then introduced, the process repeats, and so on until every new requirement has been incorporated. At no point is the full set of new requirements processed at once; the tool always sees exactly one addition on top of an otherwise stable configuration.

Formally, let P_0 denote the initial set of requirements already enforced in the network and $\{A_1, A_2, A_3, \ldots\}$ the sequence of new NSRs to be introduced. The iterative process unfolds as the following steps:

1. initial: P_0, additional: $\{A_1\}$
2. initial: $\{P_0, A_1\}$, additional: $\{A_2\}$
3. initial: $\{P_0, A_1, A_2\}$, additional: $\{A_3\}$
4. …

The total number of solver invocations equals the number of new NSRs — a linear cost — but each individual invocation is far cheaper than a single reconfiguration over the full batch, since the solver always operates considering one new requirement only. Whether the cumulative saving per step outweighs the cost of repeating the process n times is the central empirical question addressed in Chapter 5.

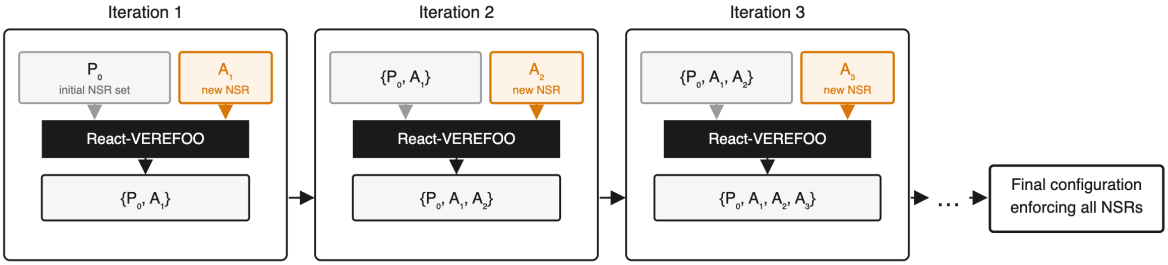


Figure 3.1: The iterative approach: each step introduces exactly one new NSR, so the solver always sees the smallest possible incremental problem.

3.2 Expected Benefits and Trade-offs

The proposed approach is expected to yield consistent performance improvements over both vanilla VEREFOO and standard React-VEREFOO, at the cost of some trade-offs that are important to acknowledge.

Expected benefits. Because each invocation of React-VEREFOO sees a problem with exactly one new requirement, the solver is always operating on the smallest possible incremental problem. This is in contrast to standard React-VEREFOO, which can receive an arbitrarily large batch of new NSRs and whose efficiency degrades as the batch grows. The proposed iterative approach removes this dependency on batch size: regardless of how many NSRs must be processed in total, each individual solver call remains minimal.

Trade-offs. The main cost of the approach is the linear number of solver invocations: where vanilla VEREFOO solves one problem and React-VEREFOO solves one (possibly large) problem, the iterative variant proposed in this thesis solves n small problems. Each invocation carries a fixed overhead due to parsing, constraint encoding, and solver initialization, which accumulates over the n iterations. Whether solving n small problems is globally faster than solving one large problem, is the central question this thesis aims to address.

A second trade-off concerns the quality of the final configuration. Vanilla VEREFOO optimises globally over the full NSR set, producing a configuration that minimises the total number of firewalls and rules jointly. The proposed iterative approach, by contrast, optimises locally at each step. In particular, a firewall placed during the first iteration to satisfy the first new NSR (defined as A_1 in previous example) may not be the best structural choice once the subsequent new requirements $(A_2, \ldots, A_n)$ are taken into account in the following iterations. Whether and to what extent this affects the quality of the final configuration in practice remains an open question, and could be the subject of further investigation.

The approach investigated in this thesis focuses exclusively on the addition of new NSRs. The case of NSR removal is not considered: when a requirement must

be revoked, the iterative strategy described here does not apply, and a separate reconfiguration procedure would be needed. Similarly, the thesis considers only the whitelisting and blacklisting policy modes supported by React-VEREFOO, and the evaluation is conducted on three specific benchmark topologies. Generalisation to other policy modes or network types is left as future work.

Chapter 4

The Iterative VEREFOO Approach

4.1 Algorithm Description

This section describes the iterative approach in detail, highlighting the structural differences with respect to VEREFOO vanilla. Both variants share the same underlying MaxSMT engine and the same formal correctness guarantees; what changes is how the set of NSRs is presented to the solver across invocations.

4.1.1 Vanilla VEREFOO

In the standard, non-incremental version of VEREFOO, all Network Security Requirements are collected into a single set and passed to the solver in one call. The algorithm proceeds as follows.

Algorithm 1 Vanilla VEREFOO

Require: Service Graph G, full NSR set $P = \{p_1, p_2, \ldots, p_n\}$
Ensure: Firewall allocation and filtering rules satisfying all $p \in P$, or UNSAT

```
1: Build Allocation Graph from G
2: result ← SolveMaxSMT(G, P)
3: if result = SAT then
4:     Extract and return firewall allocation and filtering rules
5: else
6:     return UNSAT
7: end if
```

A single MaxSMT call encodes all n requirements simultaneously. The solver searches for a globally optimal configuration, minimising the total number of allocated firewalls and rules across the entire NSR set.

At the implementation level, vanilla VEREFOO is invoked through the following constructor of `VerefooSerializer`, which takes the network topology and the algorithm variant as inputs:

Listing 4.1: Vanilla VEREFOO constructor.

```
public VerefooSerializer(NFV root, String algo) { ... }
```

4.1.2 React-VEREFOO

React-VEREFOO extends vanilla VEREFOO with the ability to reuse an existing valid configuration when the NSR set changes, rather than recomputing everything from scratch [3]. It takes two NSR sets as input: the *initial* set P_0, representing the requirements already enforced by the current configuration, and the *target* set R_t, representing the new requirements that must be incorporated. Internally, React-VEREFOO classifies every NSR into deleted (R_d), added (R_a), or kept (R_k) groups (as described in Section 2), identifies the minimal network area affected by the added requirements, and restricts the solver to that area while leaving the rest of the configuration untouched.

Algorithm 2 React-VEREFOO

Require: Current graph G, initial NSR set P_0, target NSR set R_t
Ensure: Updated firewall allocation satisfying all $p \in P_0 \cup R_t$, or UNSAT

```
1: Classify NSRs: R_k, R_a, R_d
2: Identify reconfigurable area of G based on R_a
3: result ← SolveMaxSMT(G, initial = P_0, added = R_a)
4: if result = SAT then
5:     Extract and return updated firewall allocation and filtering rules
6: else
7:     return UNSAT
8: end if
```

React-VEREFOO issues a single MaxSMT call, just like vanilla VEREFOO, but the problem it encodes is smaller: only the nodes in the reconfigurable area appear as free decision variables. The efficiency gain therefore depends directly on how large R_a is relative to the total NSR set — the smaller the added set, the fewer nodes are flagged, and the smaller the solver problem. This relationship is captured by the percentage of requirements kept: when most requirements are kept, React-VEREFOO is significantly faster than vanilla; when most requirements change, the benefit in time execution is reduced.

At the implementation level, React-VEREFOO is invoked through the same `VerefooSerializer` constructor as the vanilla variant, with an additional boolean flag that enables the reconfiguration mode:

Listing 4.2: React-VEREFOO constructor.

```
public VerefooSerializer(NFV root, String algo, Boolean react) {
    ... }
```

The iterative approach reuses the same reconfiguration primitive as React-VEREFOO, but rather than invoking it once with a large set of new requirements,

it invokes it repeatedly with $|R_a| = 1$ at each step, regardless of how many new NSRs must ultimately be incorporated.

4.1.3 Iterative VEREFOO

The iterative variant builds on React-VEREFOO as an internal primitive introducing NSRs one at a time. The key insight is that at every iteration the solver receives a minimal incremental problem with only one new requirement on top of a stable and already-verified configuration.

The algorithm maintains two evolving structures across iterations: *consideredProps*, the set of NSRs already enforced and passed as the *initial* set to React-VEREFOO and *currentGraph*, the network graph as updated by the most recent SAT result. Both are initialised before the loop begins: *consideredProps* with any pre-existing NSRs (denoted P_0), and *currentGraph* with the original input graph.

Algorithm 3 Iterative VEREFOO

Require: Service Graph G, pre-existing NSR set P_0, new NSR sequence $P = (p_1, p_2, \ldots, p_n)$
Ensure: Firewall allocation and filtering rules satisfying all $p \in P_0 \cup P$, or UNSAT at the failing step

```
Build Allocation Graph from G
consideredProps ← P_0
currentGraph ← G
for each p_i ∈ P do
    result ← SolveMaxSMT(currentGraph, initial = consideredProps, added = {p_i})
    if result = SAT then
        consideredProps ← consideredProps ∪ {p_i}
        currentGraph ← updated graph from result
    else
        return UNSAT at step i
    end if
end for
return firewall allocation and filtering rules from currentGraph
```

The total number of MaxSMT calls equals $n = |P|$, one per new NSR. Each call operates on a problem whose *added* set contains exactly one requirement — the smallest possible incremental problem. The *initial* set grows by one element at each step, shifting requirements from added to kept as the loop progresses.

4.1.4 Structural Comparison

The three variants differ along several structural dimensions, summarised in Table 4.1. The most significant difference is the number of solver invocations: both vanilla VEREFOO and React-VEREFOO issue a single MaxSMT call, while the

iterative variant issues one call per new NSR. Consequently, the scope of each call changes: vanilla encodes the full NSR set at once, React-VEREFOO processes a batch of added requirements against the existing configuration, and each iterative call encodes exactly one new requirement on top of a fixed base. Finally, vanilla VEREFOO produces a globally optimal solution, while both React-VEREFOO and the iterative variant are locally optimal — they optimise within the identified reconfigurable area but cannot guarantee global optimality across the full NSR set.

Table 4.1: Structural comparison of the three VEREFOO variants.

	Vanilla VEREFOO	**React-VEREFOO**	**Iterative VEREFOO**
Solver invocations	1	1	one per new NSR
Added NSRs per call	all at once	variable (batch)	always 1
Reuses existing config	no	yes	yes (at each step)
Optimality	globally	locally	locally (greedy)

4.2 Worked Example

To show the mechanics of the iterative approach concretely, this section traces a complete execution on a small toy example network. The scenario uses whitelisting policy mode, in which all traffic is blocked by default and only flows covered by an explicit ALLOW rule are permitted.

4.2.1 Network Topology and NSR Set

The network consists of three endpoint nodes: a *Client* (C), a *WebServer* (W), and a *Database* (D), connected in a linear chain. Traffic from C to D must therefore transit through W. Two Allocation Places — AP_1 on the C–W link and AP_2 on the W–D link — are the candidate positions where the solver may allocate firewalls.

Figure 4.1: Simple example topology. Solid circles represent endpoints; dashed rectangles are Allocation Places.

Three NSRs must be enforced, introduced one at a time in the following order:

- p_1: C $\rightarrow$ W — *reachability* (client must be able to reach the web server)
- p_2: W $\rightarrow$ D — *reachability* (web server must be able to access the database)
- p_3: C $\rightarrow$ D — *isolation* (client must not be able to reach the database directly)

The initial pre-existing set is $P_0 = \emptyset$: no firewall is allocated and no requirement is enforced at the start.

4.2.2 Iteration 1 — Introducing p_1

- $initial = \emptyset, \quad added = \{p_1\}$
- $R_k = \emptyset, \quad R_a = \{p_1\}, \quad R_d = \emptyset$

React-VEREFOO analyses the path C$\rightarrow$W and identifies AP_1 as the relevant allocation area. Because all traffic is blocked by default and p_1 requires the path C$\rightarrow$W to be permitted, the solver allocates a firewall FW_1 at AP_1 with the following filtering policy:

$$\texttt{FW}_1\ \texttt{in}\ \texttt{AP}_1\texttt{:}\ \ \texttt{ALLOW C} \rightarrow \texttt{W}\ \ \texttt{|}\ \ \texttt{default: DROP}$$

After the solver returns SAT, p_1 is moved into *consideredProps* and the graph is updated to G_1.

4.2.3 Iteration 2 — Introducing p_2

- $initial = \{p_1\}, \quad added = \{p_2\}$
- $R_k = \{p_1\}, \quad R_a = \{p_2\}, \quad R_d = \emptyset$

The reconfigurable area is now the path W$\rightarrow$D, which involves AP_2. The firewall FW_1 at AP_1 is kept untouched, as it lies outside the identified area. The solver allocates a second firewall FW_2 at AP_2:

$$\texttt{FW}_1\ \texttt{in}\ \texttt{AP}_1\texttt{:}\ \ \texttt{ALLOW C} \rightarrow \texttt{W}\ \ \texttt{|}\ \ \texttt{default: DROP}$$
$$\texttt{FW}_2\ \texttt{in}\ \texttt{AP}_2\texttt{:}\ \ \texttt{ALLOW W} \rightarrow \texttt{D}\ \ \texttt{|}\ \ \texttt{default: DROP}$$

After the execution of this iteration, p_2 is added to *consideredProps* and the graph is updated to G_2.

4.2.4 Iteration 3 — Introducing p_3

- $initial = \{p_1, p_2\}, \quad added = \{p_3\}$
- $R_k = \{p_1, p_2\}, \quad R_a = \{p_3\}, \quad R_d = \emptyset$

p_3 requires that C cannot reach D. The only path from C to D passes through AP_1. At AP_1, FW_1 has a single ALLOW rule for traffic whose destination is W; any packet from C destined for D carries a different destination address and therefore hits the default DROP action. The isolation requirement is already satisfied by the current configuration without any modification.

React-VEREFOO confirms the existing configuration as SAT and no new firewall or rule is added. p_3 is moved into *consideredProps* and the graph remains G_2.

4.2.5 Final Configuration and Summary

The final allocation consists of two firewalls, one at each allocation place, with minimal rule sets:

$\texttt{FW}_1$ `in` $\texttt{AP}_1$`:` `ALLOW` C $\rightarrow$ W | `default: DROP`

$\texttt{FW}_2$ `in` $\texttt{AP}_2$`:` `ALLOW` W $\rightarrow$ D | `default: DROP`

Table 4.2 summarises the state of all sets at each iteration.

Table 4.2: Evolution of NSR sets and PercReqKept across the three iterations.

Iter.	**Added**	R_k	R_a	R_d
1	p_1	$\emptyset$	$\{p_1\}$	$\emptyset$
2	p_2	$\{p_1\}$	$\{p_2\}$	$\emptyset$
3	p_3	$\{p_1, p_2\}$	$\{p_3\}$	$\emptyset$

Three observations follow from this toy example. First, as more requirements are incorporated, the area of the network that the solver must reconsider is smaller at each iteration: the nodes satisfying all previously introduced requirements are treated as fixed, and only the area relevant to the single new requirement is left as a free decision variable. Secondly, already-enforced requirements impose no additional solving cost since their satisfaction is guaranteed by construction from the previous step and the solver does not need to reason about them again. Finally, iteration 3 required no modification at all as the isolation requirement was already implied by the whitelisting default of FW_1, showing that the iterative approach can incorporate new NSRs with zero reconfiguration cost when the existing configuration already covers them.

These observations are specific to this toy network and cannot be generalised directly to real deployments, where larger topologies and more complex requirement interactions make the outcome of each iteration far less predictable. They

are presented here purely as a reasoning tool, to build intuition for why the iterative approach should, in principle, reduce the overall solving effort compared to a monolithic invocation. Whether this translates into a measurable gain in practice is the question addressed experimentally in Chapter 5.

Chapter 5

Implementation and Validation

5.1 Implementation

The iterative variant is implemented by modifying `VerefooSerializer`, the class that serves as the entry point to the VEREFOO tool: all invocations — vanilla, React, and now iterative — start here. Rather than passing the full set of new requirements to React-VEREFOO in a single call, the modified constructor wraps the React-VEREFOO invocation in a loop that introduces one NSR at a time. The vanilla constructor, also defined in `VerefooSerializer`, is left untouched, as are all downstream components — `VerefooProxy`, the MaxSMT encoding, and the `Translator`.

5.1.1 Entry Point and initialization

Since the iterative variant modifies the React-VEREFOO constructor, it shares the same parameters: `root` is the input `NFV` object containing the network topology and the full NSR set, `algo` selects the algorithm variant (`"AP"` for Atomic Predicates or `"MF"` for MaxFlow)[1], and `react` is the boolean flag that activates the React-VEREFOO reconfiguration.

The constructor begins by building the Allocation Graph from the input Service Graph, exactly as the other two variants do:

Listing 5.1: Allocation graph construction.

```
AllocationGraphGenerator agg = new
   AllocationGraphGenerator(root, react);
root = agg.getAllocationGraph();
```

Before the iterative loop starts, two data structures are initialized: `consideredProps` and `currentGraph`. `consideredProps` holds the requirements that have already been enforced and which will be passed as the *initial* set to React-VEREFOO at

[1] In this thesis, only the Atomic Predicates algorithm is used, as React-VEREFOO supports only the `AP` variant.

each step, while `currentGraph` holds the current state of the network graph, which is updated after every successful solver invocation.

Listing 5.2: Initialization of the iterative state.

```
List<Property> consideredProps = new ArrayList<>();
if (initialProp != null) {
    consideredProps.addAll(initialProp);
}
Graph currentGraph = g;
```

If the input `NFV` object contains a pre-existing set of already enforced requirements (`initialProp`), they are loaded into `consideredProps` at the start. This supports the general case in which the network already has a valid configuration before the first new NSR is introduced, corresponding to $P_0 \neq \emptyset$ in the algorithm description of Chapter 4.

5.1.2 Core change: the Iterative Loop

The main loop iterates over `prop`, the list of new requirements (NSRs) to incorporate. At each step, `VerefooProxy` is expects two NSR lists: `consideredProps` as the initial set of requirements already enforced, and `singleProp` as the new property to be added[2]. `VerefooProxy` then computes the difference between the two lists to derive the *added*, *kept*, and *deleted* requirement sets, as described in Chapter 2. By construction, R_a (the *added* set) always contains exactly one element.

Listing 5.3: Core of the iterative loop.

```
for (Property currentProp : prop) {
    List<Property> singleProp = new ArrayList<>();
    singleProp.add(currentProp);
    singleProp.addAll(consideredProps);

    VerefooProxy test = new VerefooProxy(
        currentGraph,
        root.getHosts(),
        root.getConnections(),
        root.getConstraints(),
        consideredProps, // initial set (already enforced)
        singleProp, // new property to be enforced
        paths,
        AlgoUsed
    );
    ...
}
```

[2] `singleProp` is built as shown in Listing 5.3 to match the input format expected by the `VerefooProxy` interface.

After each `VerefooProxy` call, the result is examined. If all the requirements are satisfied, the current requirement is added to `consideredProps` and `currentGraph` is updated to the graph produced by the solver, so that the intermediate result is propagated to the next iteration. If the solver returns UNSAT, the loop does not break immediately, but instead the current `root` is kept as the result and the failure is recorded and surfaced to the caller after the loop completes.

5.1.3 Summary of Changes

In summary, the core changes to the original React-VEREFOO is the introduction of the iterative loop: instead of passing the full set of new requirements to React-VEREFOO in a single call, the execution is broken in multiple intermediate steps, each adding exactly one requirement at a time. Between steps, the graph and the set of enforced requirements are updated, so that each React-VEREFOO invocation builds on a valid and already verified configuration. All other components — `VerefooProxy`, the MaxSMT encoding, the `Translator` — remain unchaged.

5.2 Testing

5.2.1 Test Overview

The experimental evaluation aims to assess the performance of the iterative approach against vanilla VEREFOO across different network topologies and NSR set sizes. The metric used is total execution time: the cumulative time required to reach a final valid configuration, including all intermediate React-VEREFOO invocations. Each test case was repeated over 20 runs, and the reported values represent the average over those runs.

Evaluating additional metrics, such as the number of allocated firewall instances and the number of generated filtering rules, which would quantify any loss of global optimality introduced by the greedy sequential strategy, is left as a direction for future work.

Three benchmark topologies are used, chosen to cover a spectrum from small real-world networks to large synthetic ones.

Stanford.

CESNET.

Texas 2000.

5.2.2 Test Configurations

All three test classes invoke `VerefooSerializer` with the `react = true` flag and the Atomic Predicates (`AP`) algorithm. The vanilla baseline is obtained by invoking the single-argument constructor on the same input, without the iterative loop. The configurations differ in how the network topology and NSR set are generated.

Stanford. NSRs are generated synthetically on top of the Stanford topology using `TestCaseGeneratorStanford`. The test is parameterised by three values: the number of properties (`numberPRs`), the ratio of reachability and isolation requirements (`percentPairs`), and the fraction of port-specific rules (`portSpecifics`). In this thesis, `percentPairs` and `portSpecifics` are held constant at `{0.5, 0.0}` and `0.00` respectively across all test cases, while `numberPRs` varies over {10, 20, 30, 40, 50, 60}, producing six distinct NSR set sizes. Each configuration is executed 20 times, and the reported values are the averages over those runs.

CESNET. The CESNET topology is loaded via the `TestCaseGeneratorCesnet`. Unlike Stanford, the topology itself is fixed and only the NSR set varies across test cases. Isolation requirements are not bidirectional (`isolationBidirectional = false`) and port-level constraints are disabled (`usePorts = false`). The number of NSRs is increased progressively across test runs by raising `policyNumber`, with `reachabilityPerc = 0.5`, so that approximately half of the generated NSRs are reachability properties and the other half are isolation properties. Each configuration is executed 20 times.

Texas 2000. For the Texas 2000 topology, both the network graph and the NSR set are generated synthetically by `TestCaseGeneratorTexas2000`. The topology size is controlled by two parameters: `numberUCC` (???) and `numberSS` (???). The test iterates over six predefined `(UCC, SS)` pairs — `(1,3)`, `(2,6)`, `(3,9)`, `(4,12)`, `(5,15)`, `(6,18)` — corresponding to NSR counts of 48, 96, 144, 192, 240, and 288 respectively. The topology size as well as number of NSR is varied to assess its impact on performance. Each configuration is executed 20 times, and the reported values are the averages over those runs.

Table 5.1 summarises the configuration parameters used in each test class, as read from the source code.

5.2.3 Results

Figure **??** reports the average execution times measured across the three benchmark topologies. Each data point represents the mean over 20 runs. The Y-axis uses a logarithmic scale to accommodate the wide range of values observed, particularly for larger networks.

[3]For Texas 2000, the ratio of reachability to isolation requirements is not a configurable parameter. The mix of isolation requirements and reachability properties emerges from the topology structure rather than from an explicit ratio setting.

Table 5.1: Configuration parameters for each benchmark test class (values from source code).

Parameter	**Stanford**	**CESNET**	**Texas 2000**
Algorithm	AP	AP	AP
Runs per config	20	20	20
`percentPairs` / `reachabilityPerc`	`{0.5, 0.0}`	`0.5`	—[3]
NSR set sizes	10, 20, 30, 40, 50, 60	20, 40, 60, 80, 100, 120	48, 96, 144, 192, 240, 288

5.2.4 Comparison and Discussion

Chapter 6

Conclusion and Future Work

6.1 Conclusions

6.2 Future implementation

- ordinamento degli NSR (NSR Ordering): menzionare come implementazione futura - valutazione di metriche aggiuntive come il numero di regole e firewall finali, oltre al tempo di esecuzione - test quantitativi

Bibliography

[1] D. Bringhenti, G. Marchetto, R. Sisto, and F. Valenza, "Automation for Network Security Configuration: State of the Art and Research Trends," *ACM Computing Surveys*, vol. 56, no. 3, 2023.

[2] D. Bringhenti, G. Marchetto, R. Sisto, F. Valenza, and J. Yusupov, "Automated Firewall Configuration in Virtual Networks," *IEEE Transactions on Dependable and Secure Computing*, vol. 20, pp. 1559–1576, March-April 2023.

[3] F. Pizzato, D. Bringhenti, R. Sisto, and F. Valenza, "Automatic and Optimized Firewall Reconfiguration," in *Proc. IEEE/IFIP Network Operations and Management Symposium (NOMS 2024)*, 2024.

[4] D. Bringhenti, S. Bussa, R. Sisto, and F. Valenza, "Atomizing Firewall Policies for Anomaly Analysis and Resolution," *IEEE Transactions on Dependable and Secure Computing*, vol. 22, no. 3, 2025.

[5] D. Bringhenti, S. Bussa, R. Sisto, and F. Valenza, "A Two-Fold Traffic Flow Model for Network Security Management," *IEEE Transactions on Network and Service Management*, vol. 21, no. 4, 2024.